

Axios — Deep Dive (All Important Concepts Explained Clearly)

1. What Axios is and why use it

Axios is a promise-based HTTP client for browsers and Node.js. It simplifies making HTTP requests (GET, POST, PUT, DELETE) and handling responses.

It automatically transforms JSON, supports interceptors (for auth, logging, retries), cancellation, progress tracking, and simpler error handling.

2. Installation & basic usage

`npm install axios`

Basic usage example (Promise and `async/await` forms) to make GET requests from APIs.

3. Request config — key options

Options: `method`, `url`, `baseUrl`, `params`, `data`, `headers`, `timeout`, `auth`, `responseType`, `withCredentials`, `onUploadProgress`, `signal`.

Allows customization for specific request requirements.

4. Response object

Contains `data`, `status`, `statusText`, `headers`, `config`, `request`. The `'data'` property is what you typically need.

5. Error object

Contains `error.response` (HTTP error), `error.request` (no response), `error.message` (setup or network issue). Check which exists to handle properly.

6. Instances and defaults

Create reusable clients with `axios.create({ baseUrl, timeout, headers })` for modular code. Also supports global defaults via `axios.defaults`.

7. Interceptors (request & response)

Functions that run before requests or after responses. Used for adding tokens, logging, refresh logic, etc.

8. Token refresh flow (advanced)

Implements token refresh logic and queues pending requests during refresh.

9. Cancellation

Use `AbortController` to cancel requests, especially useful in React components on unmount.

10. Upload/Download progress

Monitor file upload or download progress using `onUploadProgress` / `onDownloadProgress`.

11. Params serialization

Customize how query params are formatted (use `paramsSerializer`).

12. TransformRequest / TransformResponse

Preprocess request or response data (e.g., convert to `snake_case`, decrypt).

13. Concurrency

Use `Promise.all` or `axios.all` for multiple parallel requests.

14. TypeScript usage

Define data types using generics, and narrow error types with `axios.isAxiosError()`.

15. Retries & backoff

Implement retry logic manually or use `axios-retry` library.

16. Security & best practices

Prefer `httpOnly` cookies for tokens (`withCredentials: true`). Use CSRF tokens with Laravel. Avoid storing tokens in `localStorage`.

17. Common patterns / structure

Create `client.js` and `service` files for clean, modular API layers in React or Node apps.

18. Debugging tips

Use interceptors for logging (development only), check devtools Network tab, handle CORS properly.

19. Common code snippets

GET, POST, PUT, DELETE examples, file uploads, cancel requests, set headers globally.

20. Summary

Axios simplifies HTTP communication. Mastering configs, interceptors, and error handling gives full control over client-server interaction.